



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12 de marzo del 2026

CALIFICACIÓN: _____

ACTIVIDADES REALIZADAS

GRUA JERARQUICA

Como primera actividad se pidió completar la grúa del ejercicio en clase, dicha actividad de práctica se completó y adjunto documentación en la entrega anterior, pero se resume la finalización de esta actividad a continuación.

Archivo Window.h

Agregamos los getters para las llantas (se renombraron articulación 5 y 6, mientras que llanta 3 y 4 son getters nuevos).

```
// -----  
// CAMBIO 2: Getters para las llantas  
GLfloat getLlanta1() { return llanta1; }  
GLfloat getLlanta2() { return llanta2; }  
GLfloat getLlanta3() { return llanta3; }  
GLfloat getLlanta4() { return llanta4; }  
// -----
```

También se agregaron las nuevas variables

```
// -----  
// CAMBIO 3: Variables para almacenar articulaciones y llantas  
GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4,  
llanta1, llanta2, llanta3, llanta4;  
// -----
```

Archivo Window.cpp

Ahora, dentro de window.cpp hubo dos cambios cruciales, primero, dentro de `Window::Window(GLint windowHeight, GLint windowWidth)` se eliminó articulación 5 y 6 para colocar e inicializar a llanta 1 a 4.

```
// -----  
// CAMBIO 4: Inicializar variables para las llantas  
llanta1 = 0.0f;  
llanta2 = 0.0f;  
llanta3 = 0.0f;  
llanta4 = 0.0f;  
// -----
```

Pero el cambio más crucial fue dentro de `void Window::ManejaTeclado(...)` ya que se agregaron teclas nuevas y límites para el movimiento de los brazos de la grúa obteniendo por resultado la siguiente **TABLA 1**:

TECLA	MOVIMIENTO	LIMITE
F	Brazo 1 hacía adelante	Sup: 10.0f
V	Brazo 1 hacía atrás	Inf: -110.0f

G	Brazo 2 hacia adelante	Sup: 250.0f
B	Brazo 2 hacia atrás	Inf: -60.0f
H	Brazo 3 hacia adelante	Sup: 65.0f
N	Brazo 3 hacia atrás	Inf: -220.0f
J	Canasta hacia adelante	Sup: 180.0f
M	Canasta hacia atrás	Inf: -2.0f
K	Llanta 1 hacia adelante	-
L	Llanta 2 hacia adelante	-
I	Llanta 3 hacia adelante	-
O	Llanta 4 hacia adelante	-

```
// ===== CAMBIO 5: Controles de grua =====
// =====

// 1.- BRAZO ARTICULADO: ARTICULACIÓN 1
if (key == GLFW_KEY_F){          // Tecla F: Movimiento hacia adelante
    theWindow->articulacion1 += 5.0f;
    if (theWindow->articulacion1 > 10.0f){ // LimSup
        theWindow->articulacion1 = 10.0f;}
}
if (key == GLFW_KEY_V){          // Tecla V: Movimiento hacia atrás
    theWindow->articulacion1 -= 5.0f;
    if (theWindow->articulacion1 < -110.0f) { // LimInf
        theWindow->articulacion1 = -110.0f; }
}

// 1.- BRAZO ARTICULADO: ARTICULACIÓN 2
if (key == GLFW_KEY_G) { // Tecla G: Hacia adelante
    theWindow->articulacion2 += 5.0f;
    if (theWindow->articulacion2 > 250.0f) {
        theWindow->articulacion2 = 250.0f;}
}
if (key == GLFW_KEY_B){          // Tecla B: Hacia atrás
    theWindow->articulacion2 -= 5.0f;
    if (theWindow->articulacion2 < -60.0f) { // LimInf
        theWindow->articulacion2 = -60.0f;}
}

// 1.- BRAZO ARTICULADO: ARTICULACIÓN 3
if (key == GLFW_KEY_H){          // Tecla H: Hacia adelante
    theWindow->articulacion3 += 5.0f;
    if (theWindow->articulacion3 > 65.0f) { // LimSup
        theWindow->articulacion3 = 65.0f;}
}
if (key == GLFW_KEY_N){          // Tecla N: Hacia atrás
    theWindow->articulacion3 -= 5.0f;
    if (theWindow->articulacion3 < -220.0f){ // LimInf
        theWindow->articulacion3 = -220.0f;}
}

// 1.- BRAZO ARTICULADO: ARTICULACIÓN 4
if (key == GLFW_KEY_J){          // Tecla J: Hacia adelante
    theWindow->articulacion4 += 5.0f;
    if (theWindow->articulacion4 > 180.0f){ // LimSup
```

```

        theWindow->articulacion4 = 180.0f;}
    }
    if (key == GLFW_KEY_M){          // Tecla M: Hacia atrás
        theWindow->articulacion4 -= 5.0f;
        if (theWindow->articulacion4 < -2.0f){ // LimInf
            theWindow->articulacion4 = -2.0f;}
    }

    // =====

    // 2.- CILINDROS: Llanta 1 (Delantera Izquierda)
    if (key == GLFW_KEY_K){
        theWindow->llanta1 += 5.0f;}
    // 2.- CILINDROS: Llanta 2 (Delantera Derecha)
    if (key == GLFW_KEY_L){
        theWindow->llanta2 += 5.0f;}
    // 2.- CILINDROS: Llanta 3 (Trasera Izquierda)
    if (key == GLFW_KEY_I){
        theWindow->llanta3 += 5.0f;}
    // 2.- CILINDROS: Llanta 4 (Trasera Derecha)
    if (key == GLFW_KEY_O){
        theWindow->llanta4 += 5.0f;}
    // =====

```

Archivo main (SECCIÓN YA COMENTADA)

Regresando a nuestro main, dentro del ciclo `while (!mainWindow.getShouldClose())` comenzamos con el modelado jerárquico de la grúa, primero establecemos nuestro nodo raíz/padre como la cabina (prisma rectangular). Es importante mencionar que aquí SI inicializamos la matriz de transformación principal, y es gracias a la variable `modelaux` que podemos aplicar transformaciones, escalamientos y rotaciones jerárquicas para que los demás componentes se conecten a nuestra cabina heredando/siendo hijos de su posición global y rotación sin deformarse.

```

// *****
// ***** MODELADO JERÁRQUICO: GRUA *****
// *****

// =====
// CAMBIO 6: NODO RAÍZ: CABINA
// =====
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.0f, -4.0f));
modelaux = model; // GUARDAMOS: Centro de la cabina (padre)
model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.859f, 0.533f, 0.318f); // Naranja base
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

```

Teniendo la base, vamos a crear la base de la grúa con una pirámide cuadrangular, esta hereda de la posición de la cabina con `modelaux`. También creamos un modelo de apoyo llamado `modelBaseCab` para que de ahí colguemos nuestras cuatro llantas, siendo que a cada una se le aplica una rotación de 90° base, además de la rotación con teclado.

```
// -----
// CAMBIO 7: BASE Y LLANTAS (Heredan de la Cabina)
// -----
// ---- BASE PIRÁMIDE (Hijo de Cabina)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f)); // Bajamos la base
glm::mat4 modelBaseCab = model; // GUARDAMOS: Centro de la base (padre de llantas)
model = glm::scale(model, glm::vec3(3.0f, 1.5f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.3f, 0.3f, 0.3f); // Gris oscuro
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMeshGeometry();

// ---- LLANTAS (Hijas de la Base)
glm::vec3 escalaLlanta = glm::vec3(0.8f, 0.8f, 1.0f);
glm::vec3 colorLlanta = glm::vec3(0.188f, 0.188f, 0.184f);
// Llanta 1: Delantera Izquierda (Tecla K)
model = modelBaseCab;
model = glm::translate(model, glm::vec3(-1.0f, -0.5f, 2.3f));
model = glm::rotate(model, glm::radians(mainWindow.getLlanta1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, escalaLlanta);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
meshList[2]->RenderMeshGeometry();
// Llanta 2: Delantera Derecha (Tecla L)
model = modelBaseCab;
model = glm::translate(model, glm::vec3(1.0f, -0.5f, 2.3f));
model = glm::rotate(model, glm::radians(mainWindow.getLlanta2()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, escalaLlanta);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
meshList[2]->RenderMeshGeometry();
// Llanta 3: Trasera Izquierda (Tecla I)
model = modelBaseCab;
model = glm::translate(model, glm::vec3(-1.0f, -0.5f, -2.3f));
model = glm::rotate(model, glm::radians(mainWindow.getLlanta3()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, escalaLlanta);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
meshList[2]->RenderMeshGeometry();
// Llanta 4: Trasera Derecha (Tecla O)
model = modelBaseCab;
model = glm::translate(model, glm::vec3(1.0f, -0.5f, -2.3f));
model = glm::rotate(model, glm::radians(mainWindow.getLlanta4()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, escalaLlanta);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorLlanta));
meshList[2]->RenderMeshGeometry();
```

Un cambio extra que se realizo fue poner pequeños detalles en la grúa, tales como 3 ventanas (cubos aplanados), un contrapeso que suele tener la grúa y un motor con un cilindro en la parte delantera, para ello se instanciaron 5 modelos extra reutilizando modelaux de la cabina.

```
// -----
// CAMBIO 8: DETALLES DE LA CABINA (Heredan de la Cabina)
// -----
glm::vec3 colorVentana = glm::vec3(0.725f, 0.863f, 0.922f); // Azul vidrio
// ---- CONTRAPESO TRASERO
```

```

model = modelaux;
model = glm::translate(model, glm::vec3(2.5f, -0.5f, 0.0f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 2.8f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.388f, 0.345f, 0.314f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- VENTANA FRONTAL
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, 0.2f, 0.0f));
model = glm::scale(model, glm::vec3(0.1f, 1.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(colorVentana);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- VENTANA DERECHA
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.2f, 1.51f));
model = glm::scale(model, glm::vec3(3.5f, 1.0f, 0.1f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(colorVentana);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- VENTANA IZQUIERDA
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.2f, -1.51f));
model = glm::scale(model, glm::vec3(3.5f, 1.0f, 0.1f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(colorVentana);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- CILINDRO FRONTAL (Motor)
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, -0.7f, 0.0f)); // Justo debajo de la ventana
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.6f, 3.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.859f, 0.533f, 0.318f); // Mismo color naranja de la cabina
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();

```

Finalmente, a partir de los dos brazos de la grúa ya programados por el profesor, se buscó instanciar el tercer brazo faltante junto a su canasta. Comenzando desde la cabina, se hicieron de manera secuencial tres brazos conectados por esferas que actúan como pivotes (articulaciones). En cada paso, la pieza actual guarda su matriz de transformación, pero sin escalar y se la heredamos a la siguiente pieza. Esto incluye la parte final (la canasta), garantizando así que nuestras rotaciones aplicadas en las articulaciones base afecten a todos los nodos hijos que sigan, imitando la física real de una grúa articulada.

```

// -----
// CAMBIO 9: BRAZO ARTICULADO (Hijos)
// -----
// ---- ARTICULACIÓN 1 Y BRAZO 1 (Hijo de Cabina)
model = modelaux; // Partimos nuevamente desde la cabina
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
modelaux = model; // GUARDAMOS: Centro Brazo 1
model = glm::scale(model, glm::vec3(5.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.710f, 0.502f, 0.361f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- ARTICULACIÓN 2 (Hijo de Brazo 1)

```

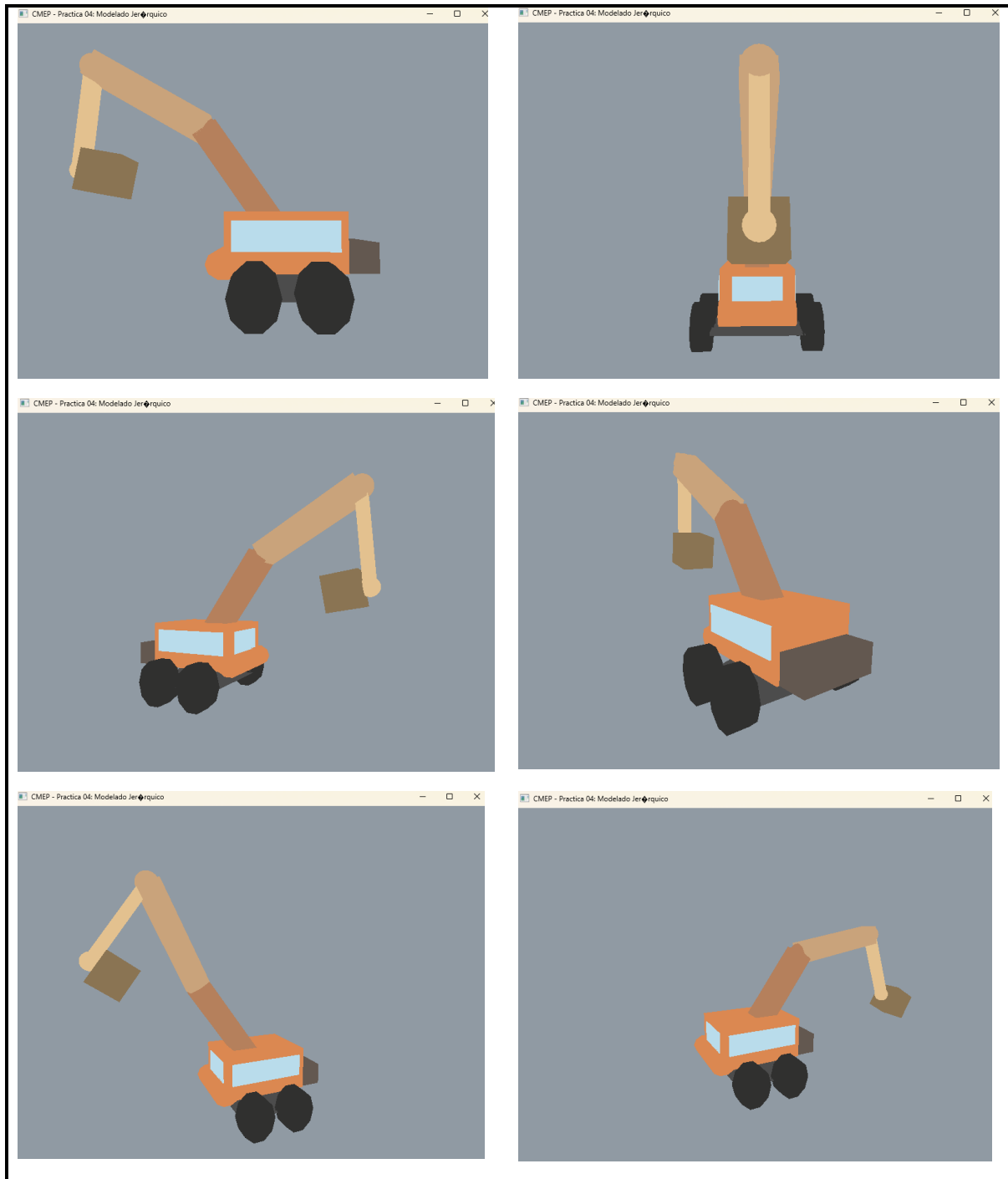
```

model = modelaux;
model = glm::translate(model, glm::vec3(2.5f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model; // GUARDAMOS: Pivote Articulación 2
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();
// ---- BRAZO 2 (Hijo de Articulación 2)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
modelaux = model; // GUARDAMOS: Centro Brazo 2
model = glm::scale(model, glm::vec3(1.0f, 5.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.788f, 0.639f, 0.482f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- ARTICULACIÓN 3 (Hijo de Brazo 2)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -2.5f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model; // GUARDAMOS: Pivote Articulación 3
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();
// ---- BRAZO 3 (Hijo de Articulación 3)
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
modelaux = model; // GUARDAMOS: Centro Brazo 3
model = glm::scale(model, glm::vec3(4.0f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.890f, 0.757f, 0.561f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
// ---- ARTICULACIÓN 4 (Hijo de Brazo 3)
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model; // GUARDAMOS: Pivote Articulación 4
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
sp.render();
// ---- CANASTA (Hijo de Articulación 4)
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model; // GUARDAMOS: Centro Canasta
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.541f, 0.455f, 0.325f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();

```

EJECUCIÓN

Para la ejecución se usa la cámara libre y las teclas WASD (para la cámara) mientras que las teclas F,V,G,B,H,N,J,M,I,O,K y L se usan para las rotaciones (*Ver tabla 1*)



Se adjunta link a drive del GIF para apreciar mejor el movimiento

https://drive.google.com/file/d/1Jd96SftVYo3eJRBqTCvwUgHqk_6mKyh2/view?usp=sharing

ANIMAL JERARQUICO

Como segundo ejercicio se pidió crear un animal 3D, instanciando cubos, pirámides, cilindros, conos o esferas. Como requisitos se pide que el animal tenga 5 patas articuladas en dos partes, una cola articulada o dos orejas articuladas, todas estas pueden moverse de forma independiente con el teclado. Para esta actividad nos inspiramos en “**CATNAP**” personaje de videojuegos con extremidades alargadas y delgadas.



Archivo Window.h

Agregamos los getters para nuestras articulaciones, tenemos un total de 11, 8 para las patas, 2 para la cola y 1 para la cabeza.

```
// -----  
// CAMBIO 1 :Articulaciones de las patas  
GLfloat getarticulacion11() { return articulacion11; }  
GLfloat getarticulacion12() { return articulacion12; }  
GLfloat getarticulacion21() { return articulacion21; }  
GLfloat getarticulacion22() { return articulacion22; }  
GLfloat getarticulacion31() { return articulacion31; }  
GLfloat getarticulacion32() { return articulacion32; }  
GLfloat getarticulacion41() { return articulacion41; }  
GLfloat getarticulacion42() { return articulacion42; }  
// CAMBIO 2: Articulaciones de la cola y cabeza  
GLfloat getarticulacion01() { return articulacion01; }  
GLfloat getarticulacion02() { return articulacion02; }  
GLfloat getarticulacion00() { return articulacion00; }  
// -----
```

También se agregaron las nuevas variables

```
// -----  
// CAMBIO 3: Variables para almacenar articulaciones  
GLfloat rotax, rotay, rotaz,  
    articulacion11, articulacion12, articulacion21, articulacion22,  
    articulacion31, articulacion32, articulacion41, articulacion42,  
    articulacion00, articulacion01, articulacion02;  
// -
```

Archivo Window.cpp

Ahora, dentro de window.cpp hubo dos cambios cruciales, primero, dentro de `Window::Window(GLint windowHeight, GLint windowWidth)` se inicializaron todas nuestras 11 articulaciones

```
// -----  
// CAMBIO 4: Inicializar variables para articulaciones  
articulacion11 = 0.0f; articulacion12 = 0.0f; // Pata 1  
articulacion21 = 0.0f; articulacion22 = 0.0f; // Pata 2  
articulacion31 = 0.0f; articulacion32 = 0.0f; // Pata 3  
articulacion41 = 0.0f; articulacion42 = 0.0f; // Pata 4  
articulacion01 = 0.0f; // Cola 1  
articulacion02 = 0.0f; // Cola 2  
articulacion00 = 0.0f; // Cabeza (Rotación)  
// -----
```

Pero el cambio más crucial fue dentro de `void Window::ManejaTeclado(...)` ya que se agregaron teclas nuevas para el movimiento de las patas, la cola y la cabeza, obteniendo por resultado la siguiente **TABLA 2**:

TECLA	PARTE	MOVIMIENTO
7	Pata 1– Delantera Izq. (Superior)	Hacia delante
8	Pata 1– Delantera Izq. (Superior)	Hacia atrás
F	Pata 1– Delantera Izq. (Inferior)	Hacia delante
V	Pata 1– Delantera Izq. (Inferior)	Hacia atrás
4	Pata 2– Delantera Der. (Superior)	Hacia delante
5	Pata 2– Delantera Der. (Superior)	Hacia atrás
G	Pata 2– Delantera Der. (Inferior)	Hacia delante
B	Pata 2– Delantera Der. (Inferior)	Hacia atrás
1	Pata 3– Trasera Izq. (Superior)	Hacia delante
2	Pata 3– Trasera Izq. (Superior)	Hacia atrás
H	Pata 3– Trasera Izq. (Inferior)	Hacia delante
N	Pata 3– Trasera Izq. (Inferior)	Hacia atrás
9	Pata 4– Trasera Der. (Superior)	Hacia delante
0	Pata 4– Trasera Der. (Superior)	Hacia atrás
J	Pata 4– Trasera Der. (Inferior)	Hacia delante
M	Pata 4– Trasera Der. (Inferior)	Hacia atrás
I	Cola (Superior)	Hacia arriba

K	Cola (Superior)	Hacia abajo
O	Cola (Inferior)	Hacia arriba
L	Cola (Inferior)	Hacia abajo
Z	Cabeza	Hacia Derecha
X	Cabeza	Hacia Izquierda

```
// =====
// CONTROLES DE CATNAP
// =====

// --- PATA 1 (Delantera Izquierda) ---
// Superior (7-8)
if (key == GLFW_KEY_7) { theWindow->articulacion11 += 2.0f; }
if (key == GLFW_KEY_8) { theWindow->articulacion11 -= 2.0f; }
// Inferior (F-V)
if (key == GLFW_KEY_F) { theWindow->articulacion12 += 2.0f; }
if (key == GLFW_KEY_V) { theWindow->articulacion12 -= 2.0f; }

// --- PATA 2 (Delantera Derecha) ---
// Superior (4-5)
if (key == GLFW_KEY_4) { theWindow->articulacion21 += 2.0f; }
if (key == GLFW_KEY_5) { theWindow->articulacion21 -= 2.0f; }
// Inferior (G-B)
if (key == GLFW_KEY_G) { theWindow->articulacion22 += 2.0f; }
if (key == GLFW_KEY_B) { theWindow->articulacion22 -= 2.0f; }

// --- PATA 3 (Trasera Izquierda) ---
// Superior (1-2)
if (key == GLFW_KEY_1) { theWindow->articulacion31 += 2.0f; }
if (key == GLFW_KEY_2) { theWindow->articulacion31 -= 2.0f; }
// Inferior (H-N)
if (key == GLFW_KEY_H) { theWindow->articulacion32 += 2.0f; }
if (key == GLFW_KEY_N) { theWindow->articulacion32 -= 2.0f; }

// --- PATA 4 (Trasera Derecha) ---
// Superior (9-0) -> Ajustado como acordamos
if (key == GLFW_KEY_9) { theWindow->articulacion41 += 2.0f; }
if (key == GLFW_KEY_0) { theWindow->articulacion41 -= 2.0f; }
// Inferior (J-M)
if (key == GLFW_KEY_J) { theWindow->articulacion42 += 2.0f; }
if (key == GLFW_KEY_M) { theWindow->articulacion42 -= 2.0f; }

// --- COLA ---
// Segmento 1 (I-K)
if (key == GLFW_KEY_I) { theWindow->articulacion01 += 2.0f; }
if (key == GLFW_KEY_K) { theWindow->articulacion01 -= 2.0f; }
// Segmento 2 (O-L)
if (key == GLFW_KEY_O) { theWindow->articulacion02 += 2.0f; }
if (key == GLFW_KEY_L) { theWindow->articulacion02 -= 2.0f; }

// --- CABEZA ---
// Lados (Z-X)
if (key == GLFW_KEY_X) { theWindow->articulacion00 += 2.0f; }
if (key == GLFW_KEY_Z) { theWindow->articulacion00 -= 2.0f; }

// =====
```

Archivo MAIN

Antes de comenzar a armar la geometría, definimos nuestra paleta de colores utilizando vectores `glm::vec3`. Aquí asignamos valores RGB específicos para el pelaje, las

articulaciones, orejas, garras y detalles, lo cual nos facilitará colorear cada primitiva geométrica más adelante enviando estas variables al shader mediante variables uniformes con la ayuda de `uniformColor`

```
// *****
// ***** MODELADO JERARQUICO DE CATNAP *****
// *****

// =====
// 1. PALETA DE COLORES DE CATNAP
// =====
glm::vec3 colorArticulacion = glm::vec3(0.369f, 0.263f, 0.431f);
glm::vec3 colorPelaje = glm::vec3(0.349f, 0.231f, 0.420f);
glm::vec3 colorOrejas = glm::vec3(0.310f, 0.224f, 0.361f);
glm::vec3 colorOrejasInteriores = glm::vec3(0.420f, 0.333f, 0.471f);
glm::vec3 colorPatasSup = glm::vec3(0.278f, 0.192f, 0.329f);
glm::vec3 colorPatasInf = glm::vec3(0.216f, 0.161f, 0.251f);
glm::vec3 colorGarras = glm::vec3(0.188f, 0.157f, 0.212f);
glm::vec3 colorNegro = glm::vec3(0.125f, 0.118f, 0.129f);
glm::vec3 colorBlanco = glm::vec3(0.950f, 0.950f, 0.950f);
```

A continuación, establecemos nuestro nodo raíz/padre, siendo este el torso de Catnap. Es importante mencionar que aquí SÍ inicializamos la matriz de transformación principal `model = glm::mat4(1.0);` y le asignamos la rotación global de la escena. Después guardamos este modelo en `modelaux` con la que podemos escalar y dibujar la geometría del torso sin afectar el original, conservando un punto de anclaje .

```
// =====
// 2. NODO RAÍZ: TORSO DE CATNAP
// =====
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 6.0f, 0.0f));
// Rotación global de la escena (Con las teclas E, R, T)
model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
glm::mat4 modelaux = model; // GUARDAMOS: Centro del torso (Nodo Padre absoluto)
// Dibujo del torso
model = glm::scale(model, glm::vec3(1.4f, 1.5f, 4.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPelaje));
sp.render(); // (Esfera estirada)
```

Para construir la cabeza, partimos de nuestro torso (`modelaux`). Nos trasladamos a la parte frontal-superior y aplicamos la primera rotación de articulación (controlada por teclado), guardando este pivote en la variable `modelCuello`. A partir de este nodo, instanciamos la cabeza y creamos un nuevo modelo de auida, (`modelCabeza`). Este último lo convertimos en un nodo padre para todos los detalles faciales: orejas exteriores e interiores, los ojos (cuencas y pupilas), la nariz y formamos su característica sonrisa superponiendo una esfera sobre otra.

```
// =====
// 3. CUELLO, CABEZA Y DETALLES FACIALES (Hijos del Torso)
// =====
// ---- ARTICULACIÓN 0 (Cuello) ----
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.8f, 3.7f));
// Articulación: Rotación de la cabeza (Teclas Z y X)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion00()), glm::vec3(0.0f, 1.0f, 0.0f));
glm::mat4 modelCuello = model; // GUARDAMOS: Pivote del cuello
// Dibujo del cuello
```

```

model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- CABEZA -----
model = modelCuello; // Hereda rotación del cuello
model = glm::translate(model, glm::vec3(0.0f, 1.5f, 1.0f));
glm::mat4 modelCabeza = model; // GUARDAMOS: Centro de la cabeza (Padre de rostro y orejas)
// Dibujo de la cabeza
model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPelaje));
sp.render();
// ----- DETALLES FACIALES: OREJAS, OJOS Y BOCA (Hijos de la cabeza) -----
// ---- OREJAS EXTERIORES -----
// Izquierda
model = modelCabeza;
model = glm::translate(model, glm::vec3(1.0f, 1.6f, 0.0f));
model = glm::rotate(model, glm::radians(-28.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.5f, 2.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorOrejas));
meshList[4] -> RenderMeshGeometry();
// Derecha
model = modelCabeza;
model = glm::translate(model, glm::vec3(-1.0f, 1.6f, 0.0f));
model = glm::rotate(model, glm::radians(28.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.5f, 2.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[4] -> RenderMeshGeometry();
// ---- OREJAS INTERIORES -----
// Izquierda
model = modelCabeza;
model = glm::translate(model, glm::vec3(1.0f, 1.6f, 0.4f));
model = glm::rotate(model, glm::radians(-28.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.8f, 1.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorOrejasInteriores)); // Aplicamos nuevo color
meshList[4] -> RenderMeshGeometry();
// Derecha
model = modelCabeza;
model = glm::translate(model, glm::vec3(-1.0f, 1.6f, 0.4f));
model = glm::rotate(model, glm::radians(28.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.8f, 1.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[4] -> RenderMeshGeometry();
// ---- BOCA (Sonrisa de media luna) -----
// Fondo negro
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.0f, -0.4f, 1.15f));
model = glm::scale(model, glm::vec3(1.1f, 0.7f, 0.20f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
sp.render();
// Máscara (Tapa la mitad superior)
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.0f, 0.1f, 1.20f));
model = glm::scale(model, glm::vec3(1.2f, 0.6f, 0.15f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPelaje));
sp.render();
// ---- OJOS (Cuencas y Pupilas) -----
// Cuenca Izquierda
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.6f, 0.4f, 1.25f));
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.1f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
sp.render();
// Pupila Izquierda
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.6f, 0.4f, 1.35f));
model = glm::scale(model, glm::vec3(0.08f, 0.08f, 0.02f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorBlanco));
sp.render();
// Cuenca Derecha
model = modelCabeza;

```

```

model = glm::translate(model, glm::vec3(-0.6f, 0.4f, 1.25f));
model = glm::scale(model, glm::vec3(0.35f, 0.35f, 0.1f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
sp.render();
// Pupa Derecha
model = modelCabeza;
model = glm::translate(model, glm::vec3(-0.6f, 0.4f, 1.35f));
model = glm::scale(model, glm::vec3(0.08f, 0.08f, 0.02f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorBlanco));
sp.render();
// ---- NARIZ (Pirámide invertida)
model = modelCabeza;
model = glm::translate(model, glm::vec3(0.0f, -0.1f, 1.35f)); // Centro de la cara
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
meshList[4]->RenderMeshGeometry();

```

Pasando a la cola, regresamos de nuevo a nuestro nodo padre principal (modelaux). Nos trasladamos a la parte posterior del torso e implementamos una articulación base (modelCola1) que nos permite mover el primer segmento de la cola y de este primer segmento nace un segundo pivote (modelCola2) con su propia articulación.

```

// =====
// 4. COLA (Hija del Torso)
// =====
// ---- ARTICULACIÓN 1 (Base) ----
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.5f, -4.5f));
// Articulación: Sube y baja (Teclas I-K)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion01()), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Inclinación natural
glm::mat4 modelCola1 = model; // GUARDAMOS: Pivote base cola
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- SEGMENTO COLA 1 ----
model = modelCola1;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasSup)); // Aplicamos nuevo color
sp.render();
// ---- ARTICULACIÓN 2 (Medio) ----
model = modelCola1; // Hereda de la base
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -6.0f));
// Articulación: Sube y baja (Teclas O-L)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion02()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelCola2 = model; // GUARDAMOS: Pivote medio cola
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- SEGMENTO COLA 2 ----
model = modelCola2;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.5f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasInf)); // Aplicamos nuevo color
sp.render();

```

Finalmente, instanciamos las cuatro patas de Catnap aplicando exactamente la misma lógica jerárquica modular para cada una. En todas partimos del torso (modelaux) para posicionar la articulación del hombro o la cadera, aplicamos su articulación

correspondiente y guardamos ese pivote. Aquí es donde este nodo padre nos permite dibujar la parte superior de la pata y, a su vez, hereda su posición a un segundo nodo para la rodilla (modelRodilla1), que se encarga de articular la parte inferior. Por último, la rodilla le hereda la posición a la planta del pie (modelPie1), el cual funciona como el último nodo padre encargado de posicionar correctamente las tres garras. Gracias a esta cadena exacta (Torso -> Hombro/Cadera -> Rodilla -> Pie -> Garras), logramos que cada extremidad se flexione de forma independiente sin que sus piezas se separen.

```
// =====
// 5. PATA 1: DELANTERA IZQUIERDA
// =====
// ---- ARTICULACION 11 (HOMBRO: Teclas 7-8) ----
model = modelaux;
model = glm::translate(model, glm::vec3(0.8f, -0.4f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelHombro1 = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA SUPERIOR 1 ----
model = modelHombro1;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 3.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasSup)); // Aplicamos nuevo color
meshList[2]->RenderMeshGeometry();
// ---- ARTICULACION 12 (RODILLA: Teclas F-V) ----
model = modelHombro1;
model = glm::translate(model, glm::vec3(0.0f, -3.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion12()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelRodilla1 = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA INFERIOR 1 ----
model = modelRodilla1;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 3.0f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasInf)); // Aplicamos nuevo color
meshList[2]->RenderMeshGeometry();
// ---- PLANTA DEL PIE 1 ----
model = modelRodilla1;
model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.4f));
glm::mat4 modelPie1 = model;
model = glm::scale(model, glm::vec3(0.8f, 0.4f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
// ---- GARRAS 1 ----
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
// Izquierda
model = modelPie1;
model = glm::translate(model, glm::vec3(0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
// Centro
model = modelPie1;
model = glm::translate(model, glm::vec3(0.0f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
// Derecha
model = modelPie1;
model = glm::translate(model, glm::vec3(-0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
```



```

// =====
// 6. PATA 2: DELANTERA DERECHA
// =====
// ---- ARTICULACION 21 (HOMBRO: Teclas 4-5) ----
model = modelaux;
model = glm::translate(model, glm::vec3(-0.8f, -0.4f, 3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion21()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelHombro2 = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA SUPERIOR 2 ----
model = modelHombro2;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 3.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasSup)); // Aplicamos nuevo color
meshList[2]->RenderMeshGeometry();
// ---- ARTICULACION 22 (RODILLA: Teclas G-B) ----
model = modelHombro2;
model = glm::translate(model, glm::vec3(0.0f, -3.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion22()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelRodilla2 = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA INFERIOR 2 ----
model = modelRodilla2;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 3.0f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasInf)); // Aplicamos nuevo color
meshList[2]->RenderMeshGeometry();
// ---- PLANTA DEL PIE 2 ----
model = modelRodilla2;
model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.4f));
glm::mat4 modelPie2 = model;
model = glm::scale(model, glm::vec3(0.8f, 0.4f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
// ---- GARRAS 2 ----
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
// Izquierda
model = modelPie2;
model = glm::translate(model, glm::vec3(0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
// Centro
model = modelPie2;
model = glm::translate(model, glm::vec3(0.0f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
// Derecha
model = modelPie2;
model = glm::translate(model, glm::vec3(-0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();

// =====
// 7. PATA 3: TRASERA IZQUIERDA
// =====
// ---- ARTICULACION 31 (CADERA: Teclas 1-2) ----
model = modelaux;
model = glm::translate(model, glm::vec3(0.8f, -0.4f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion31()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelCadera3 = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));

```



```

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA SUPERIOR 3 ----
model = modelCadera3;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 3.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasSup)); // Aplicamos nuevo color
meshList[2] -> RenderMeshGeometry();
// ---- ARTICULACION 32 (RODILLA: Teclas H-N) ----
model = modelCadera3;
model = glm::translate(model, glm::vec3(0.0f, -3.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion32()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelRodilla3 = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA INFERIOR 3 ----
model = modelRodilla3;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 3.0f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasInf)); // Aplicamos nuevo color
meshList[2] -> RenderMeshGeometry();
// ---- PLANTA DEL PIE 3 ----
model = modelRodilla3;
model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.4f));
glm::mat4 modelPie3 = model;
model = glm::scale(model, glm::vec3(0.8f, 0.4f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0] -> RenderMesh();
// ---- GARRAS 3 ----
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
// Izquierda
model = modelPie3;
model = glm::translate(model, glm::vec3(0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3] -> RenderMeshGeometry();
// Centro
model = modelPie3;
model = glm::translate(model, glm::vec3(0.0f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3] -> RenderMeshGeometry();
// Derecha
model = modelPie3;
model = glm::translate(model, glm::vec3(-0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3] -> RenderMeshGeometry();

// =====
// 8. PATA 4: TRASERA DERECHA
// =====
// ---- ARTICULACION 41 (CADERA: Teclas 9-0) ----
model = modelaux;
model = glm::translate(model, glm::vec3(-0.8f, -0.4f, -3.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion41()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelCadera4 = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA SUPERIOR 4 ----
model = modelCadera4;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.4f, 3.0f, 0.4f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasSup)); // Aplicamos nuevo color
meshList[2] -> RenderMeshGeometry();
// ---- ARTICULACION 42 (RODILLA: Teclas J-M) ----

```

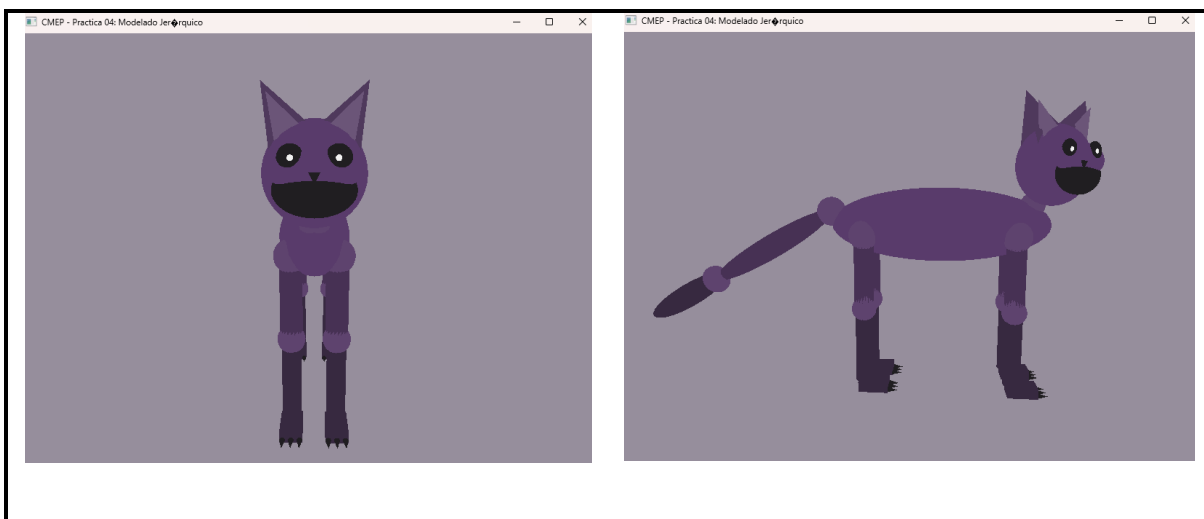
```

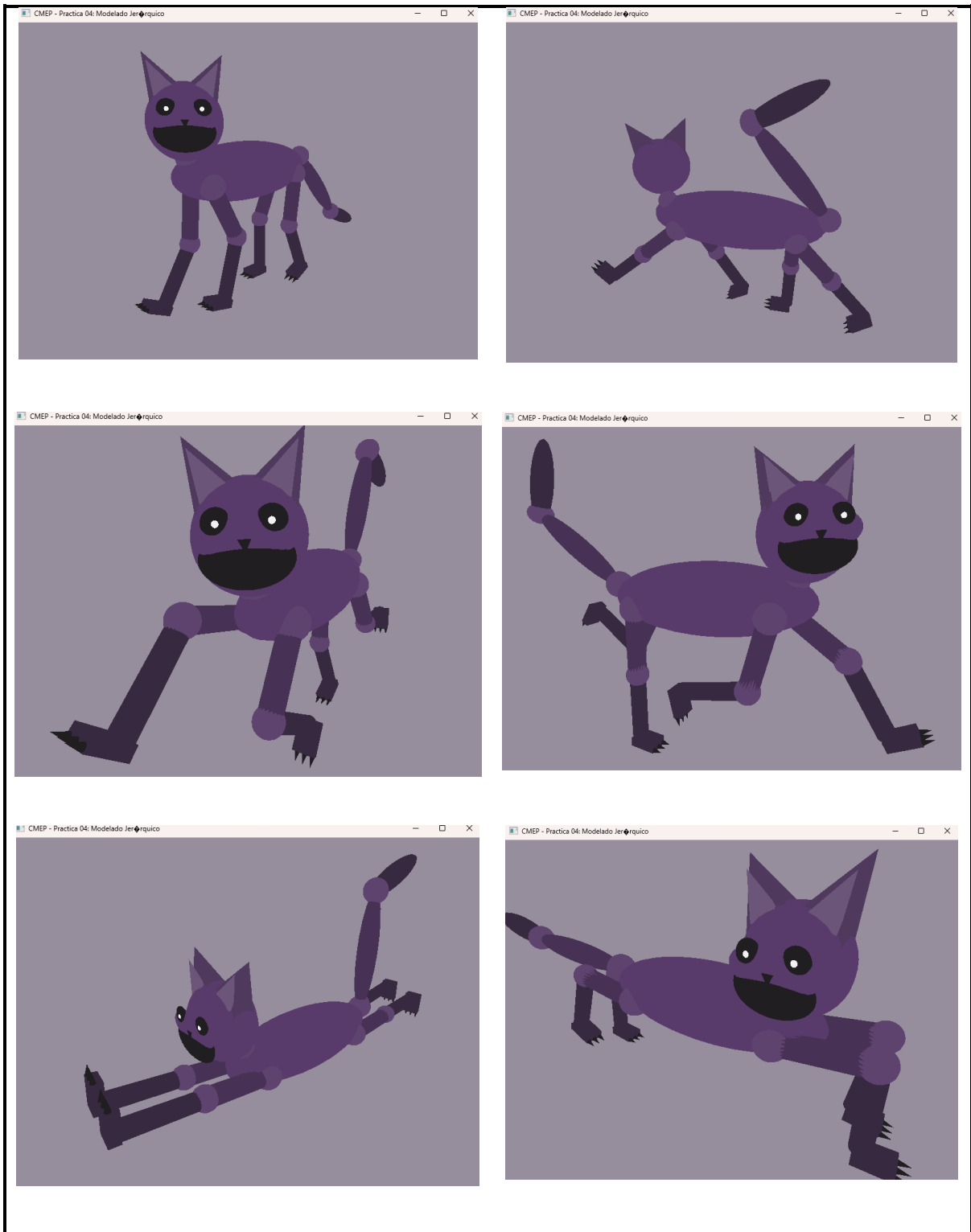
model = modelCadera4;
model = glm::translate(model, glm::vec3(0.0f, -3.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion42()), glm::vec3(1.0f, 0.0f, 0.0f));
glm::mat4 modelRodilla4 = model;
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorArticulacion));
sp.render();
// ---- PIERNA INFERIOR 4 -----
model = modelRodilla4;
model = glm::translate(model, glm::vec3(0.0f, -1.5f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 3.0f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatasInf)); // Aplicamos nuevo color
meshList[2]->RenderMeshGeometry();
// ---- PLANTA DEL PIE 4 -----
model = modelRodilla4;
model = glm::translate(model, glm::vec3(0.0f, -3.2f, 0.4f));
glm::mat4 modelPie4 = model;
model = glm::scale(model, glm::vec3(0.8f, 0.4f, 1.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0]->RenderMesh();
// ---- GARRAS 4 -----
glUniform3fv(uniformColor, 1, glm::value_ptr(colorNegro));
// Izquierda
model = modelPie4;
model = glm::translate(model, glm::vec3(0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
// Centro
model = modelPie4;
model = glm::translate(model, glm::vec3(0.0f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();
// Derecha
model = modelPie4;
model = glm::translate(model, glm::vec3(-0.3f, -0.1f, 0.8f));
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.05f, 0.4f, 0.05f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[3]->RenderMeshGeometry();

```

EJECUCIÓN

Para la ejecución se usa la cámara libre, las teclas WASD (para la cámara), las teclas E,R,T para la rotación y todas las de *TABLA 2*.

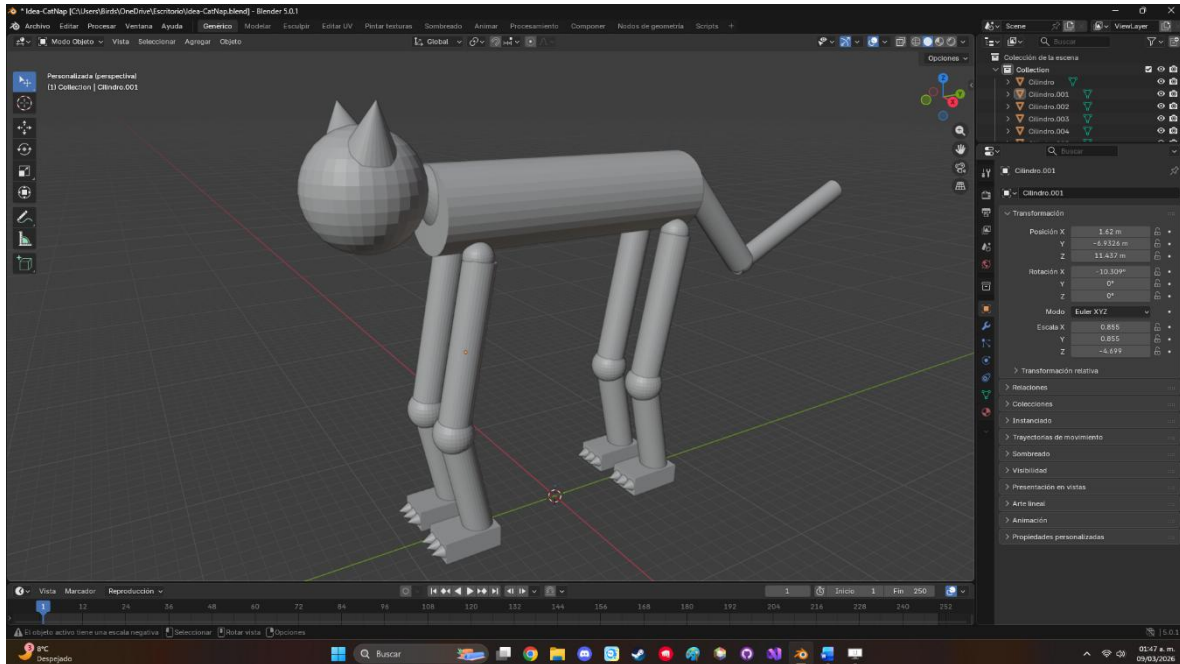




Se adjunta link a drive del GIF para apreciar mejor el movimiento
https://drive.google.com/file/d/1Jd96SftVYo3eJRBqTCvwUgHqk_6mKyh2/view?usp=sharing

PROBLEMAS PRESENTADOS

Se presentaron dificultades para definir el modelo que podría usarse para el animal jerárquico, así que se recurrió a hacer la idea base en blender primero mediante la inserción de primitivas, en su mayoría, esferas, cilindros, conos y cubos. Al final la idea o algunos aspectos fueron modificados, como el torso, la cola y las orejas, pero la base se mantuvo fiel a la idea de modelado en blender.



CONCLUSION

Sobre los ejercicios

Con los ejercicios solicitados para esta práctica se nos permitió profundizar en el concepto de modelado jerárquico mediante la implementación de dos modelos distintos: una grúa articulada y un personaje complejo inspirado en "Catnap". La complejidad técnica residió principalmente en la correcta gestión de las matrices de transformación (`glm::mat4`) y el uso de variables auxiliares (`modelaux`) para asegurar que las transformaciones de los nodos "padre" se heredaran correctamente a los "hijos" sin deformar las primitivas individuales. Además, aunque en esta ocasión no se requirió de mucha consulta externa, el trabajo fue arduo y tardado.

Comentarios generales

Durante la realización de la práctica, se presentó una dificultad inicial para visualizar la estructura del animal jerárquico directamente en código, lo que se resolvió utilizando Blender como herramienta de "prototipado" para definir la disposición de las esferas, cilindros y cubos. Como sugerencia para futuras prácticas, sería de gran ayuda contar con

una explicación más pausada sobre la acumulación de transformaciones cuando existen múltiples niveles de profundidad (como la cadena Torso -> Hombro -> Rodilla -> Pie -> Garras), ya que un error en el orden de las funciones translate o rotate puede romper la jerarquía visual del modelo como sucedió en ciertas partes de la resolución.

Cierre

Se logró vincular exitosamente la lógica de programación en C++ y OpenGL con el control por teclado para manipular estructuras articuladas complejas como lo fueron una grúa y un animal sencillo. El uso de modelado jerárquico demostró ser una técnica esencial para simular movimientos mecánicos y orgánicos realistas, garantizando que cada componente de la escena mantenga su posición relativa con su entorno.

REFERENCIAS

- **LearnOpenGL.** (2024). *Coordinate Systems and Transformations*. Recuperado el 8 de marzo de 2026, de <https://learnopengl.com/Getting-started/Coordinate-Systems>
- **Blender Foundation.** (2024). *Parenting Objects: Establishing Hierarchical Relationships*. Blender 4.2 Manual. Recuperado el 8 de marzo de 2026, de https://docs.blender.org/manual/en/latest/scene_layout/object/editing/parent.html
- **Blender Guru.** (2023). *Blender Beginner Tutorial: Modeling with Primitives*. Recuperado el 8 de marzo de 2026, de <https://www.blenderguru.com/tutorials/blender-beginner-tutorial-series>